0.1 Matrix Decomposition

Let $A \in \mathbb{R}^{m \times n}$ be the matrix that solves the system described by $Ax = b$ for $x \in \mathbb{R}^n$ and $b \in \mathbb{R}^m$.

$$A = \begin{bmatrix} a_{1,1} & a_{1,2} & \cdots & a_{1,n} \\ a_{2,1} & a_{2,2} & \cdots & a_{2,n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m,1} & a_{m,2} & \cdots & a_{m,n} \end{bmatrix}, \qquad x = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}, \qquad b = \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_m \end{bmatrix}.$$

After performing Gaussian elimination on A, U (row reduction of A) is in row echelon form if:

- All zero rows are at the bottom.
- The first nonzero entry in a row (from the left) is located to the right of the first nonzero entry in every row above.

The rank of A is the number of nonzero rows in U. We say the system $A\mathbf{x} = \mathbf{b}$ is inconsistent (i.e. no solution) if $\text{rank}(A) < \text{rank}([A\mathbf{b}])$. The system has a unique solution when $\text{rank}(A) = \text{rank}([A\mathbf{b}]) = n$ (i.e. $\text{rank}(A)$ equals the number of variables in the system). The system has infinitely many solutions when $\text{rank}(A) = \text{rank}([A\mathbf{b}])$ and $\text{rank}(A) < n$.

0.1.1 LU Decomposition

Any non-singular matrix A can be decomposed into a product of a lower triangular matrix L and an upper triangular matrix U such that $A = LU$, using the procedures of Gaussian elimination.

Theorem 1: If A can be reduced by Gaussian elimination to row echelon form using only operations of the form "add c times row j to row i" (that is, without scaling rows and without interchanging rows), then A has an LU decomposition of the form $A = LU$, where L is lower triangular and U is upper triangular.

In particular, after performing Gaussian elimination on A, the matrix U is the corresponding row echelon form of A, and L is given by

$$L = \begin{pmatrix} 1 & 0 & 0 & \cdots & 0 \\ -c_{2,1} & 1 & 0 & \cdots & 0 \\ -c_{3,1} & -c_{3,2} & 1 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ -c_{m,1} & -c_{m,2} & -c_{m,3} & \cdots c & 1 \end{pmatrix}$$

where each entry corresponds to the elementary row operation "add $c_{i,j}$ times row j to row i" performed during Gaussian elimination.

If A can be decomposed into $A = LU$, then $\mathbf{x}$ can be solved for by:

1. Let $\mathbf{y} = U\mathbf{x}$ and solve the system for $L\mathbf{y} = \mathbf{b}$.
2. Once $\mathbf{y}$ is found, the upper triangular system $U\mathbf{x} = \mathbf{y}$ can be solved with back substitution for $\mathbf{x}$.

If we transform A into U by Gaussian elimination, the lower triangular matrix$L$ can be found by performing negations of the same operations on the identity matrix in reverse order. There are other methods to find L and U, such as the Doolittle algorithm and the Crout algorithm.

Steps	
1.	Initialize L to an identity matrix, $\mathbb{I}$ of dimension $n \times n$, and $U = A$.
2.	For $i = 1, \ldots, n$ do Step 3.
3.	For $j = i + 1, \ldots, n$ do Steps 4–5.
4.	Set $l_{ji} = u_{ji}/u_{ii}$.
5.	Perform $U_j = (U_j - l_{ji}U_i)$ (where U_i, U_j represent the i and j rows of the matrix $\mathbf{U}$, respectively).

Once we have L and U, we can solve for as many right-hand side vectors $\mathbf{b}$ as desired very quickly using a two-step process. First, let $\mathbf{y} = U\mathbf{x}$ and then solve $L\mathbf{y} = \mathbf{b}$ for $\mathbf{y}$ by forward substitution.

Pseudocode for forward substitution

Steps	
1.	Set $y_1 = b_1/l_{11}$.
2.	For $i = 2, 3, \ldots, n$ do Step 3.
3.	$$y_i = \left(b_i - \sum_{j=1}^{i-1} l_{ij} y_j \right) / l_{ii}$$

Then solve $U\mathbf{x} = \mathbf{y}$ for $\mathbf{x}$ using backward substitution.

Pseudocode for backward substitution

We can also easily solve for A^{-1} and $\det(A)$ once L and U are known. For more details, see Numerical Methods for Engineers.
Sources:

- https://ubcmath.github.io/MATH307/systems/lu.html
- https://johnfoster.pge.utexas.edu/numerical-methods-book/LinearAlgebra_LU.html

Steps	
1.	Set $x_n = y_n/u_{nn}$.
2.	For $i = n-1, n-2, \ldots, 1$ do Step 3.
3.	$$x_i = \left(y_i - \sum_{j=i+1}^{n} u_{ij} x_j \right) / u_{ii}$$